Resumo — Programação Orientada a Objetos (POO) em Java

1. Introdução à Linguagem Java

História

- Desenvolvida pela Sun Microsystems (James Gosling), a partir de 1991 (nome original *Oak*).
- Lançada em 1995 como Java.
- Fundamental no desenvolvimento web; passou para a Oracle em 2010.
- Linguagem orientada a objetos.

Vantagens

- **Portabilidade**: código compilado em *bytecode*, executado pela JVM; basta ter a JVM instalada.
- **Segurança e robustez**: verificação estática de tipos, gestão automática de memória, exceções para erros em tempo de execução.
- **Ferramentas de desenvolvimento (JDK)**: compilador `javac`, interpretador `java`, ferramentas (`jar`, `javadoc`, ...).
- **APIs** (*Application Programming Interfaces*).

Modelo de programação

- Programa composto por um conjunto de **classes**; cada classe `X` deve estar num ficheiro `X.java`.
- Uma das classes deve ter o método `main()`:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

- Compilar e executar na linha de comandos:
  - `javac HelloWorld.java`
  - `java HelloWorld`

Comentários

- Linha única: `//`
- Multi-linha: `/* ... */`

```java
System.out.println("Hello World"); // This is a comment
/* The code below prints Hello World */
System.out.println("Hello World");
```

---

2. Tipos de Dados

Tipos primitivos (manipulados diretamente na VM)

Tipo	Descrição
`byte`	Inteiros de -128 a 127

Tipo	Descrição
`short`	Inteiros de -32 768 a 32 767
`int`	Inteiros de -2 147 483 648 a 2 147 483 647
`long`	Inteiros de -9 223 372 036 854 775 808 a 9 223 372 036 854 775 807
`float`	Fracionários (6–7 dígitos decimais)
`double`	Fracionários (15–16 dígitos decimais)
`boolean`	`true` / `false`
`char`	Um caráter/letra ou valor ASCII
`String`	Texto (objeto)

```java
int myNum = 5;
float myFloatNum = 5.99f;
char myLetter = 'D';
boolean myBool = true;
String myText = "Hello";
```

Exemplo de atribuição

```java
boolean flag = true;
char ch = 'A';
byte b = 12;
short s = 24;
int i = 257;
long l = 2147483648L; // sem o 'L' é interpretado como int
float f = 3.1415f;     // sem o 'f' é interpretado como double
double d = 2.1828;
```

---

3. Casting

- **Widening (automático)**: conversão de tipo mais pequeno para maior.

```java
int myInt = 9;
double myDouble = myInt; // int -> double
System.out.println(myInt);    // 9
System.out.println(myDouble); // 9.0
```

- **Narrowing (manual)**: colocar o tipo entre parênteses.

```java
double myDouble = 9.78d;
int myInt = (int) myDouble; // double -> int
System.out.println(myDouble); // 9.78
System.out.println(myInt);    // 9
```

---

4. Operadores

Aritméticos

Operador	Nome	Descrição
+	Adição	Soma dois valores
-	Subtração	Subtrai um valor de outro
*	Multiplicação	Multiplica dois valores
/	Divisão	Divide um valor por outro
%	Módulo	Resto da divisão
++	Incremento	Aumenta em 1
--	Decremento	Diminui em 1

- Divisão inteira vs. decimal:

```
int a = 10, b = 3;
System.out.println(a / b); // 3 (inteira)
double c = 10.0d, d = 3.0d;
System.out.println(c / d); // 3.333...
```

Atribuição

`x = 5`, `x += 3` (x = x+3), `x -= 3`, `x *= 3`, `x /= 3`, `x %= 3`, `x &= 3`, `x |= 3`, `x ^= 3`, `x >>= 3`, `x <<= 3`.

Comparação e lógicos

Comparação: ==, !=, >, <, >=, <=.

Lógicos: && (AND), || (OR), ! (NOT).

Precedência (da maior para a menor)

1. Parênteses
2. *, /, %
3. +, -
4. <, >, >=, <=
5. ==, !=
6. &&
7. ||
8. = (atribuição)

---

5. Entrada e Saída

Saída

```
System.out.println("Hello World!");
System.out.println("I am learning Java.");
```

Entrada (Scanner)

```java
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        System.out.println("Enter username");
        String userName = myObj.nextLine();
        System.out.println("Username is: " + userName);
    }
}
```

Métodos do Scanner

Método	Descrição
`nextBoolean()`	lê boolean
`nextByte()`	lê byte
`nextDouble()`	lê double
`nextFloat()`	lê float
`nextInt()`	lê int
`nextLine()`	lê String
`nextLong()`	lê long
`nextShort()`	lê short

---

6. Strings em Java

Declaração

- Strings são objetos da classe `String`.

```java
String s = new String("Exemplo");
String x = "Curso de Java";
String str = new String(); // objeto string nulo
```

Métodos principais

- `int length()` — número de caracteres.

```java
String s = new String("Isaias C. Boratti");
int tam = s.length(); // 17
```

- `char charAt(int p)` — caráter na posição `p` (primeira posição é 0).

```java
String x = "Exemplo";
char c = x.charAt(2); // 'e'
```

- `boolean equals(String s)` — compara se dois strings são iguais.

```
String s1 = new String("Curso Java");
boolean b = s1.equals("Curso JDK"); // false
```

- `int compareTo(String s)` — compara strings: 0 se iguais, 1 se maior, -1 se menor.

```
String s1 = new String("Joao");
String s2 = "Joana";
int k = s1.compareTo(s2); // -1
```

- `int indexOf(char c)` / `indexOf(char c, int pos)` — posição da primeira ocorrência (ou -1 se não existir).

```
String s = "Maria da Silva";
int k = s.indexOf('a');         // 1
k = s.indexOf('x');             // -1
k = s.indexOf('a', 5);          // 7
```

- `String substring(int inicio, int fim)` — copia do início até fim-1.

```
String s1 = "Curso de POO - Java";
String s2 = s1.substring(9, 12); // "POO"
```

- `toUpperCase()` / `toLowerCase()` — converte para maiúsculas/minúsculas.

```
String s1 = "Exemplo";
String s3 = s1.toUpperCase(); // "EXEMPLO"
```

- `trim()` — remove espaços no início e fim.
- `concat()` / operador + — concatenação.

```
String firstName = "John";
String lastName = "Doe";
System.out.println(firstName + " " + lastName);
System.out.println(firstName.concat(lastName));
```

Classe `Character`

Métodos estáticos para manipulação de caracteres:

- `isDigit(char)` — se é dígito.
- `isLetter(char)` — se é letra.
- `isLowerCase(char)` / `isUpperCase(char)` — se é minúscula/maiúscula.
- `toUpperCase(char)` / `toLowerCase(char)` — converte.

---

7. Condições e Ciclos

`if`, `else`, `else if`

- `if` executa um bloco quando a condição é verdadeira.
- `else` executa quando a condição do `if` é falsa.
- `else if` especifica uma nova condição se a primeira for falsa.

```java
if (itRains) {
    bringUmbrella();
} else if (itIsSunny) {
    wearSunglasses();
} else {
    goOutsideNormally();
}
```

Operador ternário (short-hand if/else)

Substitui várias linhas por uma única; útil para `if/else` simples.

```java
String result = (x > y) ? "x maior" : "y maior";
```

`switch`

Seleciona um de vários blocos a executar, em vez de múltiplos `if/else`.

```java
switch (choice) {
    case 1: System.out.println("Pizza"); break;
    case 2: System.out.println("Burger"); break;
    case 3: System.out.println("Pasta"); break;
    default: System.out.println("Nada");
}
```

Ciclos (Loops)

- `while` — executa enquanto a condição for verdadeira.
- `do/while` — executa o bloco uma vez antes de verificar a condição; depois repete enquanto for verdadeira.
- `for` — quando se sabe exatamente quantas vezes iterar.

```java
for (int i = 0; i < 5; i++) {
    System.out.println(i); // 0 a 4
}
```

- Ciclo `for` aninhado (loop dentro de loop).
- `for-each` — percorre elementos de um array (ou outra estrutura), sem contador.

```java
for (String car : cars) {
    System.out.println(car);
}
```

`break` e `continue`

- `break` — sai do ciclo (ou do `switch`).
- `continue` — salta a iteração atual e continua com a seguinte.

```java
for (int i = 0; i < 10; i++) {
    if (i == 4) break;      // para no 4
    if (i == 4) continue;   // salta o 4
```

```
    System.out.println(i);
}
```

Arrays

- Armazenam vários valores numa única variável.

```
String[] cars = {"Volvo", "BMW", "Ford"};
System.out.println(cars[0]); // "Volvo"
```